

Partitioned Hash Join with Duplicates

An OpenMP Study

Gabriele Marsili
SPM a.y. 2025/26

1 Introduction

This report describes the OpenMP implementation of the partitioned hash join developed in Module 2. The algorithm is unchanged: R and S are partitioned through a common key mapping and a local join is performed independently on each partition. The objective of the study is to assess which OpenMP constructs are appropriate for this kernel under both balanced and skewed workloads, and to compare the resulting performance with the `std::thread` implementation of Module 2.

Two parallel variants were developed. They share the same three-phase structure, the same hash-table layout and the same mapping function, but each phase is parallelised end-to-end with a different OpenMP construct: the loop variant uses worksharing `parallel for` loops throughout, the task variant emits explicit `omp task` blocks (with Longest Processing Time First ordering on the join). Keeping the construct consistent across phases isolates the cost of the construct itself, rather than mixing two parallelism models inside the same run. All experiments were carried out on a node of the SPM cluster.

2 Algorithm and OpenMP Strategies

2.1 Three-phase Structure

Given relations R and S of size N_R and N_S , each tuple carries a key k which is mapped to partition $p = k \bmod P$ with $P = 128$. Since P is a power of two, the equivalent bitmask form $k \& (P - 1)$ is used. Each relation is processed in three phases:

1. **Histogram** – count tuples per partition.
2. **Scatter** – compute exclusive prefix-sum offsets and write tuples into contiguous per-partition buffers.
3. **Join** – on each partition p , build a hash table on R_p and probe it with the tuples of S_p .

The per-partition hash table is the `FlatCountMap` carried over from Module 2: open-addressing with linear probing, count-at-build, multiplicity accumulation at probe time. Its capacity is set to the smallest power of two $\geq 2|R_p|$, which keeps the load factor below 50% and bounds the expected probe length to a small constant.

2.2 Loop-level Variant

The loop variant follows the canonical `parallel for` pattern. Each thread maintains a private histogram in a 64-byte aligned buffer; the merge into the global histogram and the exclusive prefix sum are executed by a single thread inside an `#pragma omp single` block. The scatter phase is a `parallel for` with `schedule(static)`, since the per-tuple work is constant. The join phase is then expressed as

```
#pragma omp parallel for schedule(dynamic,1) \
    default(none) shared(P, Rpart, Spart) \
    reduction(+: join_count, checksum1, checksum2)
for (uint32_t pid = 0; pid < P; ++pid) {
    JoinResult r = join_one_partition(Rpart, Spart, pid);
    join_count += r.join_count;
    checksum1 += r.checksum1;
    checksum2 += r.checksum2;
}
```

The choice of `dynamic,1` addresses the irregular per-iteration cost. Under the skew of Section 3 a hot partition holds $\approx 22.6\%$ of the records and a cold one $\approx 0.078\%$, a ratio of $\sim 290\times$ in per-partition work, so any static distribution leaves the thread that picks up the heavy partitions on the critical path. The runtime dispatch of `dynamic,1` hands out one partition at a time and lets an idle thread grab a cold partition while the hot ones are still in flight; its overhead is small at the granularity of a partition, since the sweep of Section 5.5 shows all policies within 1% on uniform input, where the imbalance penalty disappears. The `reduction(+:...)` clause covers the three `uint64_t` scalars that accumulate the result.

2.3 Task Variant with LPT

In the task variant histogram and scatter are emitted as T explicit tasks per phase, one per chunk $[(N \cdot t)/T, (N \cdot (t+1))/T)$, which reproduces the distribution of `schedule(static)` but goes through the task queue: on a regular workload the variant pays the task-creation overhead with no balancing benefit, and the comparison surfaces it. Each task captures its index t and addresses the per-task slot `local_hists[t]` (histogram) or `cursors[t]` (scatter), so the offsets remain consistent regardless of which thread the runtime picks up the task on. Two `taskgroup` blocks bracket histogram and scatter so that the sequential merge only starts once every count has landed. The join is expressed as an explicit task loop:

```
#pragma omp parallel default(none) \
```

```

    shared(P, order, Rpart, Spart, thr_results, ...)
{
    #pragma omp single nowait
    for (uint32_t idx = 0; idx < P; ++idx) {
        uint32_t pid = order[idx]; // LPT order
        #pragma omp task firstprivate(pid) \
            shared(Rpart, Spart, thr_results)
        { /* cold: build+probe; hot: probe split
            into a nested taskgroup (Sec. 2.4) */ }
    }
}

```

The `nowait` clause is required: without it the $T - 1$ workers would wait at the implicit barrier of `single` until task emission completes, instead of consuming tasks while the master is still creating them.

The `order` array drives the scheduling. Partitions are sorted by decreasing $|R_p| + |S_p|$ before submission, which is the Longest Processing Time First (LPT) heuristic¹, a classical greedy for makespan minimisation when independent tasks of variable size are dispatched to identical workers. On this problem ($P=128$, $T \leq 32$) it has two practical effects: heavy partitions enter the queue first and are therefore unlikely to land on the critical path at the end of the run, while small partitions remain available to fill the gaps left by the heavy ones. The sum $|R_p| + |S_p|$ is an appropriate cost proxy because the per-partition cost is linear in the size of its two inputs.

All tasks are tied (the OpenMP default). Each body reads `omp_get_thread_num()` once and uses the returned index locally to write into `thr_results`; no task scheduling point separates the read from its use, so the index is stable under tied semantics without further care, and untied would offer no advantage at the granularity of these tasks.

The reduction uses a manually padded array `PaddedResult thr_results[T]`, declared with `alignas(64)` and validated by a `static_assert`, so per-thread slots sit on distinct cache lines and false sharing is removed. Synchronisation between phases differs by variant: the loop variant relies on the implicit barriers of the worksharing constructs; the task variant on the implicit `taskwait` of its two `taskgroup` blocks.

2.4 Targeted Optimisations

Three additional optimisations were applied on top of the baseline strategy described above. They follow directly from the cost model of the kernel and from the structural bound observed on skewed input.

Software prefetching in scatter and probe. Both loops have a sequential read stream, which the hardware prefetcher covers, and a random-access stream, which it does not. Two `__builtin_prefetch` calls mask the miss latency on the random side: in the scatter, the destination slot `part.data[cursors[tid][pid_next]]` is prefetched 12 iterations ahead; in the probe, the hash slot `tbl.slots[slot_of(k_next)]` is prefetched 8 iterations ahead. The distances leave enough work between

prefetch and use to cover the $L1 \rightarrow L2$ latency in the scatter and the $L2 \rightarrow L3$ latency in the heavier probe iteration.

Intra-partition parallelism for hot partitions.

The asymptotic skewed efficiency $\leq H/T$ is a structural bound: no external scheduling, including LPT, can extract more parallelism than the number of heavy partitions H . The bound is broken by exposing parallelism *within* a hot partition. The build phase still runs sequentially because concurrent updates on `FlatCountMap` would require atomic counters, but the probe is read-only on the table and parallelises naturally. A partition is classified as hot when its weight $|R_p| + |S_p|$ exceeds eight times the average $\bar{w}$; its probe is split into $K = \min(T, \lfloor w/\bar{w} \rfloor)$ sub-tasks (with a lower clamp to 2). The sub-tasks are emitted inside a `taskgroup` so that the shared table outlives all readers. The probe of each hot partition can therefore be served by up to T cooperating threads, which is what restores the parallelism that LPT alone could not extract.

NUMA placement of partition buffers (attempted).

A parallel `schedule(static)` zero-init on the partition buffers was added to spread their pages across the two NUMA nodes via Linux first-touch. The step is a no-op as written: `Record` has an in-class `key{}` initialiser, so `vector::resize` already touches every page on the calling thread, pinning the buffers to one NUMA node. A correct fix needs an uninitialised allocator; this is a known limitation.

3 Skewed Workload

The skewed dataset is produced by a Bernoulli mixture model. With probability $\rho = 0.9$ the next key is sampled from a hot pool of $H = 4$ keys, chosen so that they fall on four distinct partitions; with probability $1 - \rho = 0.1$ the key is drawn uniformly from $[0, \text{max_key})$. With ($\rho=0.9$, $H=4$) the model concentrates 90% of the probes on only 4 partitions out of 128; the hot-to-cold ratio is then a structural consequence of the choice rather than a tuning knob. The expected fraction of records assigned to a single hot partition is

$$f_{\text{hot}} = \frac{\rho}{H} + \frac{1 - \rho}{P} = \frac{0.9}{4} + \frac{0.1}{128} \approx 0.226,$$

while a cold partition receives $f_{\text{cold}} \approx 0.00078$. The two values are a direct consequence of the choice of (ρ, H, P) and produce the order-of-magnitude gap between hot and cold partitions that the experiments below probe.

The generator is driven by a `splitmix64` RNG with the seed exposed on the command line, which makes runs reproducible bit-by-bit. The `hot_count` parameter is clamped to P to prevent a non-terminating loop in the case where more hot partitions are requested than exist.

4 Validation

Two complementary mechanisms are used. For small inputs ($N_R, N_S \leq 500$) both `hashjoin_seq` and

¹R. L. Graham, *Bounds on multiprocessing timing anomalies*, SIAM J. Appl. Math. 17(2):416–429, 1969; LPT is shown to achieve makespan within $4/3 - 1/(3m)$ of the optimum on m machines.

`hashjoin_omp` run an in-process *naive* $O(|R| \cdot |S|)$ verifier (`include/verifier.hpp`) that performs a double loop over all pairs and emits a `naive.verify=PASS` line when the triple matches the algorithm’s output; the verifier is invoked *outside* the timed region. For larger inputs the script `scripts/validate.sh` sweeps modes $\times$ workloads $\times$ thread counts $\in \{1, 2, 4, 8, 16\}$ at $N_R=10^6$, $N_S=2 \cdot 10^6$. The validation range stops at $T=16$ since the SMT points only re-exercise the same code paths; they are kept for the performance evaluation, which spans $T \in \{1, 2, 4, 8, 16, 20, 32\}$. The script compares the triple (`join.count`, `checksum1`, `checksum2`) against a reference: the sequential baseline for uniform input and the parallel run at $T = 1$ in the same mode for skewed input (the sequential implementation does not generate skewed data). As an additional check, the loop and task variants at $T = 4$ are required to produce identical triples on the same input.

The checksums are constructed to be invariant under task ordering: each match contributes `splitmix64(k) · m` added modulo 2^{64} , so the total is independent of the order in which tasks complete. All combinations pass on the cluster.

5 Experimental Evaluation

5.1 Setup

All experiments are run on a single compute node of *spm-cluster*, allocated through SLURM with `--exclusive` and `--cpus-per-task=32`: dual-socket Intel Xeon E5-2640 v2 at 2.00 GHz, 8 cores per socket, 2 hardware threads per core (32 logical CPUs over 16 physical cores), 128 GB RAM, two NUMA nodes. The same hardware and SLURM allocation policy were used for the Module 2 measurements reported in Section 5.6, so the cross-module comparison is apples-to-apples. No multi-node or distributed execution is used. The build uses GCC with OpenMP 4.5 and `-O3 -march=native`, compiled on the compute node so the native ISA matches the execution target. Unless stated otherwise, the parameters are $N_R = 10^7$, $N_S = 2 \cdot 10^7$, $P = 128$, seed 42, with each cell averaged over three repetitions. Affinity is fixed via `OMP_PROC_BIND=close` and `OMP_PLACES=cores`.

5.2 Strong Scaling

Figure 1 reports the speedup at $T \in \{1, 2, 4, 8, 16, 20, 32\}$, covering the full physical-core range of the node ($T = 16$), one mid-SMT point ($T = 20$), and full SMT saturation ($T = 32$). Total times are listed in Table 1.

On uniform input the task variant is slower than the loop by up to 27% at physical saturation (0.075 s vs 0.095 s at $T=16$), with the gap narrowing in the SMT range. The breakdown of Section 5.4 localises the overhead in histogram and scatter: the per-tuple cost is regular and the task model adds dispatch and per-task allocation work that the worksharing `for`-loop does not pay, while on the join the two variants stay within noise. The loop speedup grows up to $T=16$ ($\sim 7.5\times$, efficiency 47%), degrades at $T=20$ where the team straddles physical and

T	loop unif.	task unif.	loop skew	task skew
1	0.559	0.540	0.506	0.506
2	0.297	0.291	0.269	0.266
4	0.164	0.180	0.167	0.147
8	0.112	0.127	0.110	0.094
16	0.075	0.095	0.103	0.094
20	0.096	0.098	0.108	0.099
32	0.089	0.095	0.107	0.105

Table 1: Mean total time [s], $N_R=10^7$, $N_S=2 \cdot 10^7$, $P=128$.

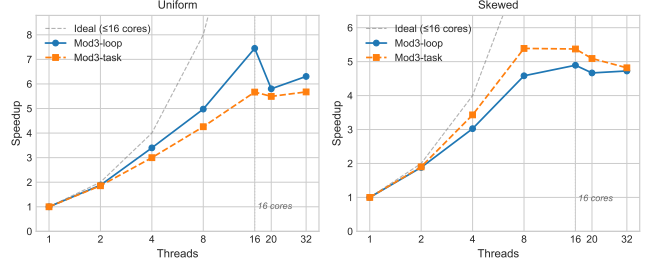


Figure 1: Strong scaling. Speedup is $T(1)/T(p)$ averaged over three repetitions. The dashed line indicates the ideal.

SMT cores, and partly recovers at $T=32$ where the kernel is already bandwidth-bound. With $T_{\text{seq}}=0.802$ s (Table 2), the absolute speedup at $T=16$ is $10.69\times$ for the loop variant and $8.42\times$ for the task variant.

On skewed input the picture inverts (Figure 2). At $T = 4$ the task variant completes in 0.147 s against 0.167 s for the loop, a $\sim 14\%$ advantage from LPT submission of the heavy partitions. The gap peaks at $T = 8$ (0.094 s vs 0.110 s, $\sim 18\%$) as the intra-partition split engages and narrows to $\sim 10\%$ at $T = 16$ (physical saturation): the join phase at $T = 16$ runs in 60.1 ms for the task variant against 73.2 ms for the loop, a $\sim 18\%$ reduction on the dominant phase. The hot-partition split (Section 2.4) lets up to four cooperating threads work inside each hot partition, which bypasses the H/T ceiling. The efficiency at $T = 16$ on skewed input is $\sim 34\%$ for the task variant against $\sim 31\%$ for the loop; the loop stays close to the ceiling because no external scheduling can manufacture parallelism between independent partitions when only four of them carry the load.

5.3 Weak Scaling

N_R scales proportionally with T , with $2 \cdot 10^6$ tuples per thread (Figure 3).

Uniform weak scaling holds within one socket: from $T=1$ to $T=8$ the loop variant grows from 0.132 s to 0.166 s (eff. $\sim 79\%$) and the task variant from 0.131 s to 0.192 s ($\sim 68\%$, with the gap absorbing the per-phase task-creation cost). Once both sockets are engaged the efficiency of the loop variant drops to $\sim 59\%$ at $T = 16$, 33% at $T = 20$, and 25% at $T = 32$; the task variant follows a similar curve, 43%, 31%, 24%. The dominant factor is memory bandwidth: the scatter phase reads and writes a working set that grows linearly with T , and the dual-socket aggregate bandwidth saturates before the core count does. Skewed weak scaling degrades more sharply because increasing N_R also scales the load

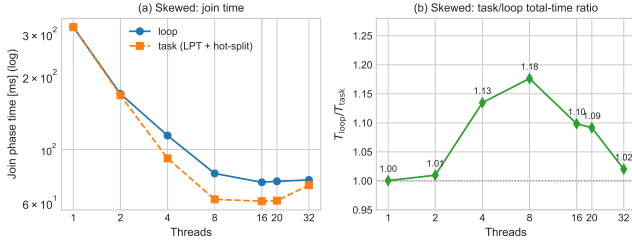


Figure 2: Skewed input. (a) Join phase time for loop and task vs T : the gap is the visible footprint of the intra-partition split. (b) Total-time ratio $T_{\text{loop}}/T_{\text{task}}$, peaking around 1.18 at $T=8$ as the intra-partition split engages and narrowing to 1.02 at $T=32$ as both variants converge to the bandwidth ceiling.

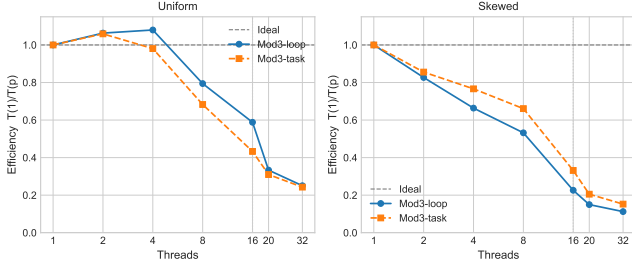


Figure 3: Weak scaling. Efficiency is $T(1)/T(p)$ with the input scaled linearly with the thread count.

on the hot partitions in proportion; the intra-partition split mitigates this on the probe but cannot eliminate the sequential cost of the per-partition build. Skewed efficiency at $T = 16$ is approximately 23% for the loop variant and 33% for the task variant.

5.4 Phase Breakdown

The breakdown in Figure 4 reflects the memory-bound nature of the kernel and surfaces the loop-vs-task trade-off at the phase level. Under uniform input the join is the dominant phase of the loop variant ($\sim 66\%$ of the total at $T=8$, $\sim 57\%$ at $T=16$, $\sim 51\%$ at $T=20$). The task variant compresses the join share to 43–58% across the same range because histogram and scatter pay the per-task dispatch cost; at $T=8$ uniform the task histogram takes 14 ms against 9 ms for the loop, while the join is roughly identical at ~ 73 ms. Under skewed input the imbalance is on the join: histogram and scatter scale similarly between the two variants because their per-tuple work does not depend on the partition distribution. At $T=16$ skewed the join accounts for $\sim 71\%$ of the total in the loop variant and $\sim 64\%$ in the task variant; at $T=20$ the gap is similar (68% vs 62%), reflecting the redistribution of probe load across threads inside each hot partition. The breakdown is not extended to $T=32$ because the SMT regime is already covered in the strong and weak scaling figures.

5.5 Schedule Sensitivity

Figure 5 shows the join time at $T = 8$ for five schedule policies on both workloads. On uniform input all policies fall within $\sim 1\%$ of each other: with $P/T = 16$ every

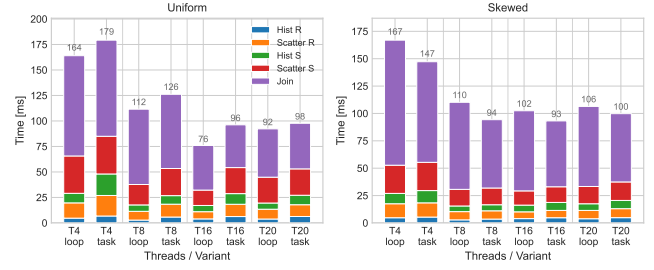


Figure 4: Phase breakdown for $T \in \{4, 8, 16, 20\}$, loop and task variants. Uniform workload on the left, skewed on the right. $T=1$ is omitted because it visually dominates and obscures the parallel regime.

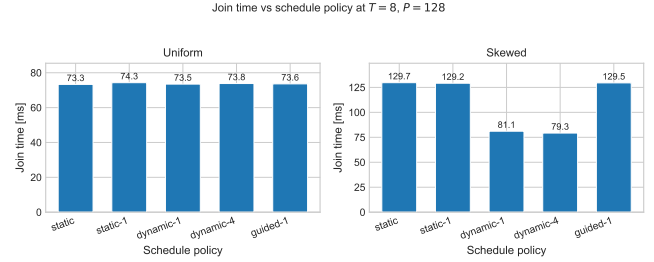


Figure 5: Join time at $T = 8$ across five schedule policies, uniform (left) and skewed (right). On uniform all policies fall within $\sim 1\%$; on skewed the two **dynamic** policies are $\sim 38\%$ faster than **static/guided**.

thread gets the same per-tuple workload regardless of how partitions are distributed. On skewed input **static**, **static,1** and **guided,1** cluster around 130 ms on the join while the two **dynamic** policies drop to ~ 80 ms, a $\sim 38\%$ reduction: **static** assigns the four hot partitions to whichever threads own their iteration range and the thread that draws all the heavy work becomes the critical path, whereas **dynamic** reassigns partitions on demand and lets idle threads steal from the heavy ones. **dynamic,4** and **dynamic,1** are within noise, so the smaller chunk size has no measurable cost at this granularity.

The skewed result confirms the choice of **dynamic,1** as the default: under skew it is the only schedule among the five that does not pay a ~ 60 ms penalty on the heavy partitions, and on uniform input it ties with the others, so it is never worse than the alternatives across the regimes observed.

5.6 Comparison with Module 2

The Module 2 implementation uses a persistent `std::thread` pool synchronised across phases by a `std::barrier` (C++20): each phase ends at the barrier, whose completion function performs the sequential merge of the per-thread histograms before the team proceeds together. Algorithm, input and node are identical to Module 3 (same SLURM allocation policy, same E5-2640 v2 hardware), so the comparison isolates the synchronisation model and the optimisations of Section 2.4. Figure 6 reports time and speedup against T for the uniform workload over the full range $T \in \{1, 2, 4, 8, 16, 20, 32\}$, matching the granularity of the Module 2 evaluation.

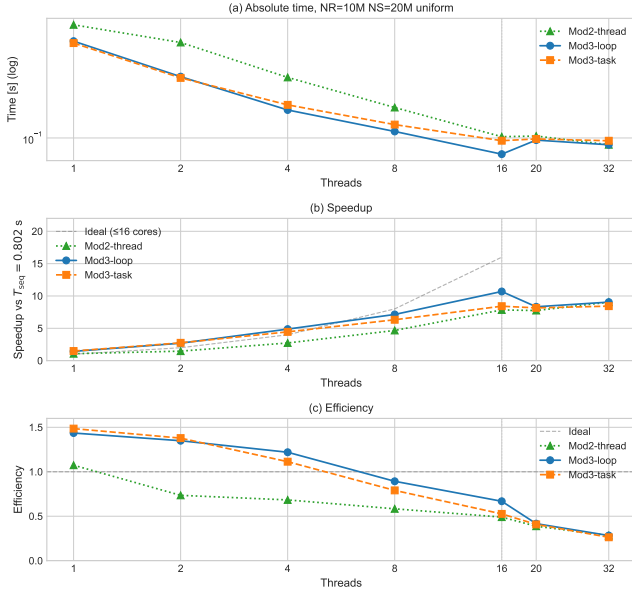


Figure 6: Module 2 `std::thread` versus Module 3 OpenMP, uniform input. (a) absolute time, (b) speedup against the shared sequential baseline, (c) efficiency = speedup/ T over the full thread range $T \in \{1, \dots, 32\}$. The vertical dotted line marks $T=16$, physical-core saturation.

T	$T(p)$ [s]			Speedup		
	Mod2	M3 loop	M3 task	Mod2	M3 loop	M3 task
seq	0.802			1.00		
1	0.747	0.559	0.540	1.07	1.44	1.49
2	0.546	0.297	0.291	1.47	2.70	2.76
4	0.293	0.164	0.180	2.74	4.89	4.46
8	0.172	0.112	0.127	4.66	7.16	6.32
16	0.102	0.075	0.095	7.86	10.69	8.44
20	0.103	0.096	0.098	7.79	8.35	8.18
32	0.089	0.089	0.095	9.01	9.01	8.44

Table 2: Module 2 `std::thread` vs Module 3 OpenMP, uniform $N_R=10^7$, $N_S=2 \cdot 10^7$, both runs on the same E5-2640 v2 SLURM allocation. The first row is the shared sequential baseline (0.802 s, current `hashjoin_seq` mean of 3 runs); the speedup columns are $T_{\text{seq}}/T(p)$. The optimisations of Section 2.4 apply to Module 3 only.

Table 2 reports the numbers at $T = 16$, $T = 20$ (physical-core saturation), and $T = 32$ (SMT range).

Both OpenMP variants outperform the `std::thread` version up to physical-core saturation, after which Module 2 catches up and at $T=32$ matches the loop variant. At $T=8$ the loop variant is $\sim 1.54\times$ faster than Module 2 and the task variant $\sim 1.35\times$; at $T=16$ the gap is $1.36\times$ for loop (0.075 s vs 0.102 s) and $1.07\times$ for task. Two phases account for most of the absolute gap. The first is the scatter: at $T=8$ the combined scatter $R+S$ takes ~ 29 ms in the loop variant against ~ 85 ms in Module 2, a difference of ~ 56 ms attributable to the software prefetch on the random-access destination slot. The second is the histogram: at $T=8$ it takes ~ 9 ms in the loop variant against ~ 35 ms in Module 2, where the `std::barrier` completion function serialises the histogram merge while the OpenMP loop variant performs the same merge in an `omp single` block whose cost is

partly hidden by the next worksharing region.

The behaviour at and beyond physical saturation differs. The loop variant peaks at $T=16$ ($10.69\times$, efficiency $\sim 67\%$) and then loses ground: at $T=20$ the team straddles physical and SMT cores and the speedup falls to $8.35\times$, recovering only partially at $T=32$ ($9.01\times$). Module 2 keeps improving with SMT, from $7.86\times$ at $T=16$ to $9.01\times$ at $T=32$, because its barrier overhead is amortised over a larger team. The two implementations meet at $T=32$, where memory bandwidth is the binding constraint and the synchronisation primitive becomes second-order.

The loop variant sits above unity for $T \leq 4$ ($\text{eff} \approx 1.22$ at $T=4$), which is not genuine super-linear scaling: the optimisations of Section 2.4 are present in the parallel binary but not in `hashjoin_seq`, so already at $T=1$ the OpenMP run is $\sim 1.44\times$ faster than the sequential baseline (Table 2). The numerator of the efficiency therefore carries a constant offset that decays as T grows and the synchronisation cost takes over. The task variant shows the same pattern but with a smaller offset because the task overhead on histogram and scatter consumes part of the gain ($\text{eff} \approx 1.12$ at $T=4$).

6 Discussion

On uniform input the loop variant is the better choice: its worksharing for-loops carry no task overhead on the regular phases, and the residual gap with the task variant grows with T , from $\sim 10\%$ at $T=4$ to $\sim 27\%$ at $T=16$. On skewed input the picture inverts: the task variant is faster from $T=4$ onwards, with an advantage that peaks at $\sim 18\%$ at $T=8$ as the intra-partition split engages and narrows to $\sim 10\%$ at physical saturation. The schedule-policy study shows that on uniform input all five policies fall within $\sim 1\%$, but on skewed input `dynamic` is $\sim 38\%$ faster than `static` and `guided`, which is what justifies `dynamic,1` as the default. Up to physical-core saturation both OpenMP variants outperform the Module 2 `std::thread` version against the shared sequential baseline: at $T=16$ the loop variant reaches $10.69\times$ speedup and the task variant $8.44\times$, against $7.86\times$ for Module 2; at $T=32$ the three implementations converge (9.01 , 8.44 and $9.01\times$ respectively) as memory bandwidth becomes the binding constraint.

Table 3 consolidates the headline numbers at the operating points of interest.

Metric	Uniform (loop)	Skewed (loop)	Skewed (task)
Best T	16	16	8
$T(p)$ [s]	0.075	0.103	0.094
Speedup vs T_{seq}	10.69	7.79	8.55
Efficiency $T(1)/T(p)/p$ at best T	47%	31%	67%
$T_{\text{loop}}/T_{\text{task}}$ at $T=8$	—	1.18	

Table 3: Summary of best operating points. The shared sequential baseline is $T_{\text{seq}}=0.802$ s. Efficiency is the within-implementation parallel efficiency $T(1)/(p \cdot T(p))$.

The OpenMP loop variant is faster than Module 2 up to physical-core saturation; the two converge at $T=32$, where memory bandwidth binds and the synchronisation primitive becomes second-order.